

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 5

Выполнил:

Сокович Степан

Группа К3344

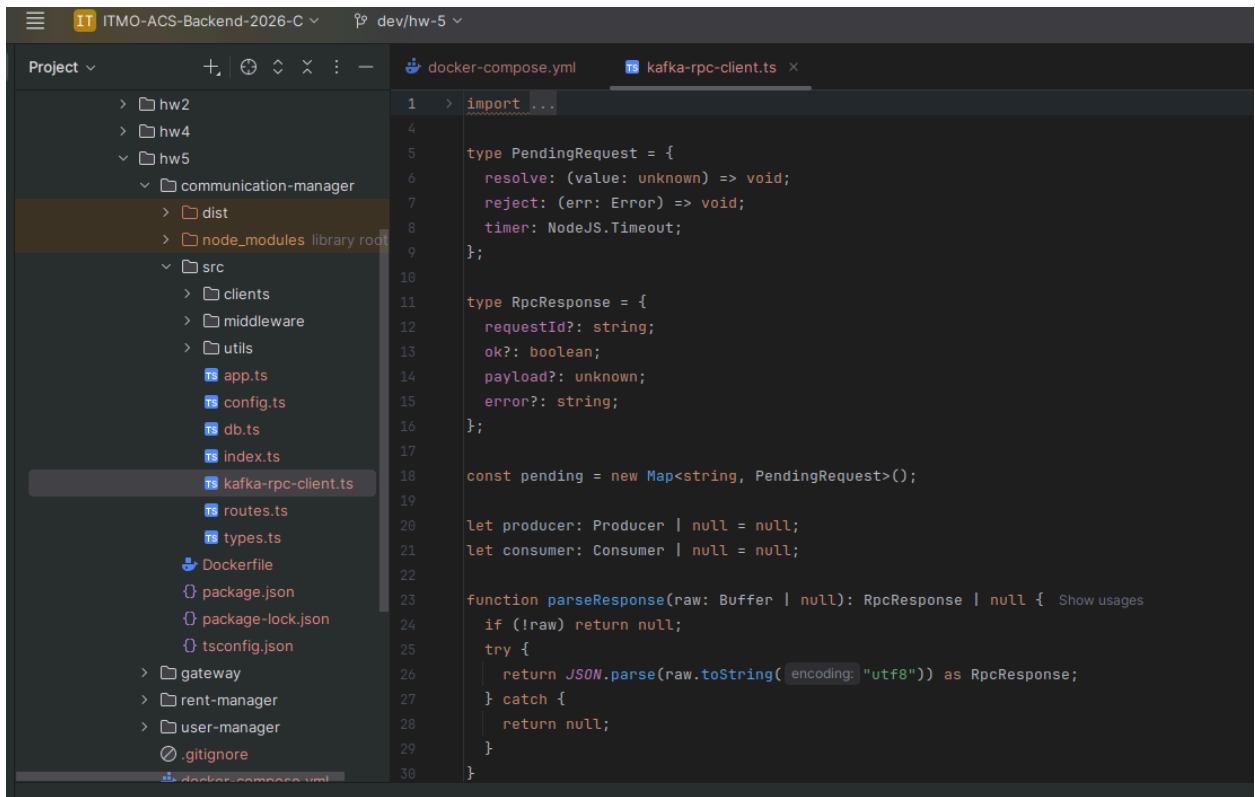
Проверил:

Добряков Д. И.

Задание:

подключить и настроить rabbitMQ/kafka;
реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Подменим существующее взаимодействие сервисов по http 1.1 обращением в кафку через грс-клиента к ней.



The screenshot shows an IDE window with a project structure on the left and a TypeScript file named `kafka-rpc-client.ts` open in the editor. The project structure includes a `communication-manager` directory with a `dist` subdirectory and a `node_modules` directory. The `kafka-rpc-client.ts` file contains the following code:

```
1 > import ...
4
5 type PendingRequest = {
6   resolve: (value: unknown) => void;
7   reject: (err: Error) => void;
8   timer: NodeJS.Timeout;
9 };
10
11 type RpcResponse = {
12   requestId?: string;
13   ok?: boolean;
14   payload?: unknown;
15   error?: string;
16 };
17
18 const pending = new Map<string, PendingRequest>();
19
20 let producer: Producer | null = null;
21 let consumer: Consumer | null = null;
22
23 function parseResponse(raw: Buffer | null): RpcResponse | null { Show usages
24   if (!raw) return null;
25   try {
26     return JSON.parse(raw.toString( encoding: "utf8")) as RpcResponse;
27   } catch {
28     return null;
29   }
30 }
```

Аналогично сделаем для каждого сервиса

Поправим существующий докер компоуз:

1) добавим создание кафки

```
services:
  kafka:
    image: bitnami/kafka:latest
    ports:
      - "9092:9092"
    environment:
      ALLOW_PLAINTEXT_LISTENER: "yes"
      KAFKA_CFG_NODE_ID: 1
      KAFKA_CFG_PROCESS_ROLES: controller,broker
      KAFKA_CFG_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CFG_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_CFG_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CFG_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_CFG_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_CFG_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_CFG_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE: "true"
      KAFKA_KRAFT_CLUSTER_ID: MkU30EBNTcwNTJENDM2Qk
    postgres-user:
```

2) прокинем как env-параметр адрес брокера для сервисов

```
1 services:
80   rent-manager:
81     build: ./rent-manager
82     depends_on:
83       kafka:
84         condition: service_started
85     postgres-rent:
86       condition: service_healthy
87     user-manager:
88       condition: service_started
89     ports:
90       - "3002:3002"
91     environment:
92       PORT: "3002"
93       DATABASE_URL: postgres://rent:rent@postgres-rent:5432/rentdb
94       USER_MANAGER_URL: http://user-manager:3001
95       INTERNAL_SERVICE_TOKEN: dev-internal-token
96       JWT_SECRET: dev-jwt-secret-change-me
97       KAFKA_BROKERS: kafka:9092
98
99   communication-manager:
100     build: ./communication-manager
101     depends_on:
102       kafka:
103         condition: service_started
104     postgres-comm:
```

Вывод: мы научились переводить микросервисную архитектуру с синхронного взаимодействия на асинхронное путём интеграции в неё Kafka